



Simulating discrete-time systems

- When simulating discrete-time state-space models, we have (at least) two options.
 - There is a direct approach where we write the simulation code ourselves;
 - Also, Octave has two functions in its “control” package that help us simulate these models easily.
 - As with continuous-time models, the first of these is “ss”, which creates a state-space object from A , B , C , and D matrices.
 - The second is “lsim”, which simulates a state-space object for given input conditions and an optional initial state.
- This lesson will show examples of both approaches, applied to the NCP, NCV, and CT models that were introduced for continuous-time simulations in Lesson 1.2.2.
- But first, we need to convert these continuous-time state-space models to equivalent discrete-time models.



Converting the nearly-constant-position model

- We might be interested to simulate a discrete-time version of the continuous-time nearly-constant-position model.
- Recall that in continuous time, the 2-d NCP model is:¹

$$\begin{aligned}\dot{x}(t) &= 0_{2 \times 2}x(t) + I_2 w(t) \\ z(t) &= x(t) + v(t).\end{aligned}$$

- In discrete time, we must then have (where Δt is the sample period):

$$\begin{aligned}x_{k+1} &= e^{0\Delta t}x_k + \left(\int_0^{\Delta t} e^{0\sigma} d\sigma\right) I w_k \\ z_k &= x_k + v_k.\end{aligned}$$

¹We are treating random input $w(t)$ in the same way we have discussed treating deterministic input $u(t)$ here. As you will learn next week, this assumption is somewhat incorrect, but is “okay for now.”



Scaled discrete-time NCP model

- To compute A_d , note that $e^{0\Delta t} = I$, which is perhaps most easily confirmed using the infinite-series expansion for e^{At} .
- To compute B_d , note that we cannot use the simplified method $B_d = A^{-1}(A_d - I)B$ since A is not invertible.
 - Instead, we use the longer-form integration method, where $\int_0^{\Delta t} I d\sigma = I\Delta t$.
- Therefore, we arrive at the discrete-time form of the NCP model:

$$\begin{aligned}x_{k+1} &= x_k + \Delta t w_k \\ z_k &= x_k + v_k.\end{aligned}$$

- Note, w_k is often scaled vis-à-vis $w(t)$ so that another commonly seen form is:

$$\begin{aligned}x_{k+1} &= x_k + w_k \\ z_k &= x_k + v_k.\end{aligned}$$



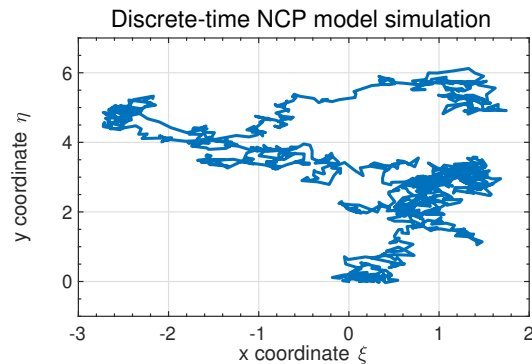
Time (dynamic) response using toolbox

- Let's first consider simulating an NCP model using "lsim".
- The figure on the right is the output from simulating the Octave code below, which uses the toolbox methods.

```
% Simulate the NCP model using lsim
A = eye(2); Bw = eye(2);
C = eye(2); D = zeros(2);
maxT = 1000; % # sim. steps
w = 0.1*randn(2,maxT); % random input
v = 0.01*randn(2,maxT);

ncp = ss(A,Bw,C,D,-1); % unspecified dT
z = lsim(ncp,w)+v';
plot(z(:,1),z(:,2));
```

- Trajectory starts at (0,0), randomly moves from there as w pushes object.

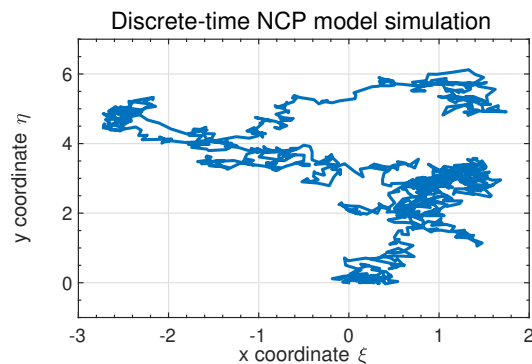


Time (dynamic) response using direct method

- In discrete-time we can implement the model without "lsim".
- We use a "for" loop to implement the state-equation recursion directly.

```
% Simulate the NCP model directly
% Assume A,Bw,C,D,maxT,w,v defined
x = zeros(2,maxT); % storage for x
x(:,1)=[0;0]; % initial posn.
for k=2:maxT % simulate model
    x(:,k)=A*x(:,k-1)+w(:,k-1);
end
z = C*x + v;
plot(z(1,:),z(2,:));
```

- This code produced the figure on the right when using the same w and v as prior slide; results are indistinguishable.



Converting the nearly-constant-velocity model

- We now consider a discrete-time version of the continuous-time nearly-constant-velocity model.
- Recall the 2-d continuous-time state equation:

$$\dot{x}(t) = \underbrace{\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}}_A x(t) + \underbrace{\begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix}}_B w(t),$$

where we are again treating random input $w(t)$ in the same way we have discussed treating deterministic input $u(t)$ here.

- We will need to compute $A_d = e^{A\Delta t}$ and also $B_d = \int_0^{\Delta t} e^{A\sigma} B d\sigma$.



Computing NCV A_d

- The discrete-time A matrix is $A_d = e^{A\Delta t}$:

$$A_d = \mathcal{L}^{-1} \left\{ (sI - A)^{-1} \right\} \Big|_{t=\Delta t} = \mathcal{L}^{-1} \left\{ \begin{bmatrix} s & -1 & 0 & 0 \\ 0 & s & 0 & 0 \\ 0 & 0 & s & -1 \\ 0 & 0 & 0 & s \end{bmatrix}^{-1} \right\} \Big|_{t=\Delta t}$$

$$= \mathcal{L}^{-1} \left\{ \begin{bmatrix} \frac{1}{s} & \frac{1}{s^2} & 0 & 0 \\ 0 & \frac{1}{s} & 0 & 0 \\ 0 & 0 & \frac{1}{s} & \frac{1}{s^2} \\ 0 & 0 & 0 & \frac{1}{s} \end{bmatrix} \right\} \Big|_{t=\Delta t} = \begin{bmatrix} 1 & t & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & t \\ 0 & 0 & 0 & 1 \end{bmatrix} \Big|_{t=\Delta t} = \begin{bmatrix} 1 & \Delta t & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \Delta t \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

- This can be verified in Octave using the “symbolic” package,

```
syms dT % in Octave, first do: pkg load symbolic
A = [0 1 0 0; 0 0 0 0; 0 0 0 1; 0 0 0 0];
Ad = expm(A*dT)
```



Computing NCV B_d

- Note that A is not invertible, so we cannot use the “simple” method to find B_d ; instead, we use the more general integration approach:

$$B_d = \int_0^{\Delta t} e^{A\sigma} B \, d\sigma = \begin{bmatrix} \Delta t^2/2 & 0 \\ \Delta t & 0 \\ 0 & \Delta t^2/2 \\ 0 & \Delta t \end{bmatrix}.$$

- This can also be verified in Octave using the “symbolic” package,

```
syms sigma dT % in Octave, first do: pkg load symbolic
A = [0 1 0 0; 0 0 0 0; 0 0 0 1; 0 0 0 0];
B = [0 0; 1 0; 0 0; 0 1];
z = expm(A*sigma);
Bd = int(z,0,dT)*B
```



Rescaling the discrete-time NCV model

- Again, we often state the discrete-time NCV model in terms of a 4-vector w_k having rescaled components.
- So, the overall discrete-time NCV model is

$$x_{k+1} = \begin{bmatrix} 1 & \Delta t & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \Delta t \\ 0 & 0 & 0 & 1 \end{bmatrix} x_k + w_k$$

$$z_k = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} x_k + v_k.$$

- Remembering the meaning of the x_k components, this is stating (what we expect!):

$$\xi_k = \xi_{k-1} + \Delta t \dot{\xi}_{k-1} + \text{noise}$$

$$\eta_k = \eta_{k-1} + \Delta t \dot{\eta}_{k-1} + \text{noise}.$$

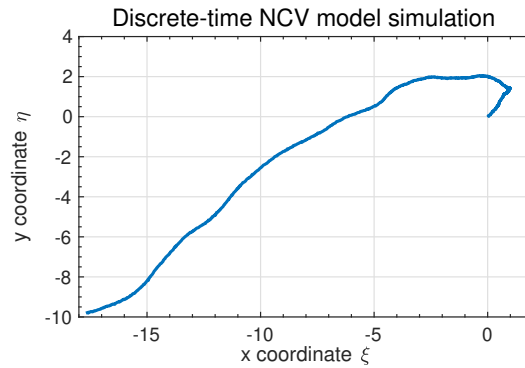


Time (dynamic) response using toolbox

- Let's first consider simulating an NCP model using "lsim".
- Figure on the right shows the output from using the toolbox methods.

```
%% Simulate the NCV model using lsim
maxT = 1000;           % # sim. steps
dT = 0.1;              % sample period
A = [1 dT 0 0; 0 1 0 0;
      0 0 1 dT; 0 0 0 1];
Bw = [dT^2/2 0; dT 0; 0 dT^2/2; 0 dT];
C = [1 0 0 0; 0 0 1 0]; D = zeros(2);
w = 0.1*randn(2,maxT); % random input
v = 0.01*randn(2,maxT);
x0 = [0;0.1;0;0.1];   % init state

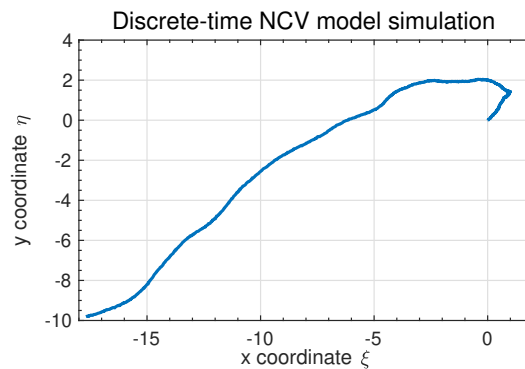
ncv = ss(A,Bw,C,D,dT);
z = lsim(ncv,w,[],x0)+v';
plot(z(:,1),z(:,2));
```



Time (dynamic) response using direct method

- In discrete-time we can implement the model without "lsim".
- We again use a "for" loop to implement the state-equation recursion directly.

```
%% Simulate the NCV model directly
% Assume A,Bw,C,D,maxT,x0,w,v defined
x = zeros(4,maxT); % storage for x
x(:,1) = x0;       % init state
for k=2:maxT       % simulate model
    x(:,k)=A*x(:,k-1)+Bw*w(:,k-1);
end
z = C*x + v;
plot(z(1,:),z(2,:));
```



- This code produced the figure on the right when using the same w and v as prior slide; results are indistinguishable.



Converting the coordinated-turn model

- Similarly, it can be shown that the discrete-time coordinated turn model in 2-d is:

$$x_k = \begin{bmatrix} 1 & \sin(\Omega \Delta t)/\Omega & 0 & (\cos(\Omega \Delta t) - 1)/\Omega \\ 0 & \cos(\Omega \Delta t) & 0 & -\sin(\Omega \Delta t) \\ 0 & (1 - \cos(\Omega \Delta t))/\Omega & 1 & \sin(\Omega \Delta t)/\Omega \\ 0 & \sin(\Omega \Delta t) & 0 & \cos(\Omega \Delta t) \end{bmatrix} x_{k-1} + \begin{bmatrix} (1 - \cos(\Omega \Delta t))/\Omega^2 & (\sin(\Omega \Delta t) - \Omega \Delta t)/\Omega^2 \\ \sin(\Omega \Delta t)/\Omega & (\cos(\Omega \Delta t) - 1)/\Omega \\ (\Omega \Delta t - \sin(\Omega \Delta t))/\Omega^2 & (1 - \cos(\Omega \Delta t))/\Omega^2 \\ (1 - \cos(\Omega \Delta t))/\Omega & \sin(\Omega \Delta t)/\Omega \end{bmatrix} w_{k-1}$$

$$z_k = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} x_k + v_k.$$



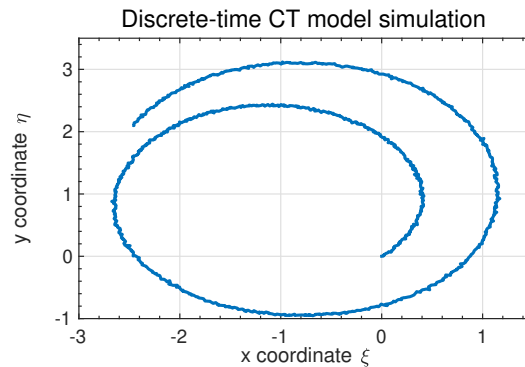
Time (dynamic) response using toolbox

- Let's first consider simulating an CT model using "lsim".

```
%% Simulate the CT model using lsim
maxT = 1000; % # sim. steps
w = 0.01*randn(2,maxT); % rand input
v = 0.01*randn(2,maxT);
x0 = [0;0.1;0;0.1]; % init state

dT = 0.1; W = 0.1; WT = W*dT;
sW = sin(WT); cW = cos(WT);
A = [1 sW/W 0 (cW-1)/W; 0 cW 0 -sW;
     0 (1-cW)/W 1 sW/W; 0 sW 0 cW];
Bw = [(1-cW)/W^2, (sW-WT)/W^2;
      sW/W, (cW-1)/W;
      (WT-sW)/W^2, (1-cW)/W^2;
      (1-cW)/W, sW/W];
C = [1 0 0 0; 0 0 1 0]; D = zeros(2);

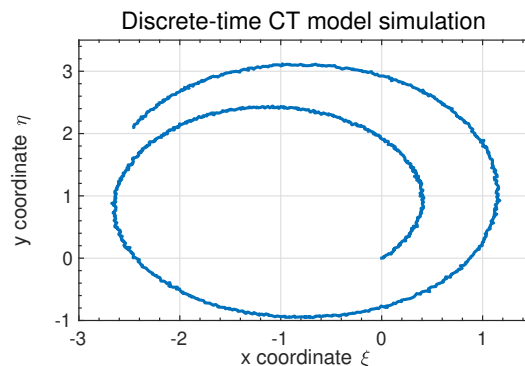
ct = ss(A,Bw,C,D,dT);
z = lsim(ct,w,[],x0)+v';
plot(z(:,1),z(:,2));
```



Time (dynamic) response using direct method

- In discrete-time we can implement the model without "lsim".
- We again use a "for" loop to implement the state-equation recursion directly.

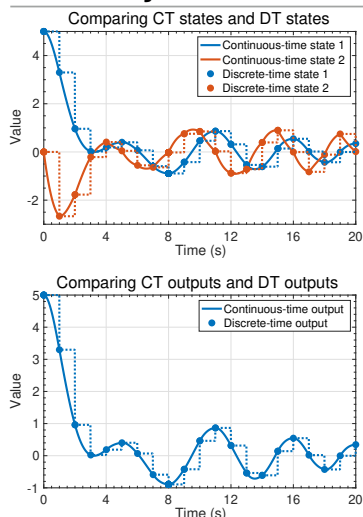
```
%% Simulate the CT model directly
% Assume A,Bw,C,D,maxT,x0,w,v defined
x = zeros(4,maxT); % storage for x
x(:,1) = x0; % init state
for k=2:maxT % simulate model
    x(:,k) = A*x(:,k-1) + Bw*w(:,k-1);
end
z = C*x + v;
plot(z(1,:),z(2,:));
```



- This code produced the figure on the right when using the same w and v as prior slide; results are indistinguishable.



Summary



- In this lesson, you learned two different ways to simulate discrete-time models in Octave: toolbox and direct.
- Both methods produce identical results; which one you use is mainly a matter of preference.
- You also learned how to convert NCP, NCV, and CT continuous-time models into discrete-time models.
- Note that the continuous-time models and discrete-time models produce identical computations at the sample points, but different values in-between sample points (technically, the discrete-time state and output are not defined in-between sample points).